

07 — DevOps Engineer

[← Powrót do README](#) (../README.md) | [← Frontend Developer](#) (../frontend-developer.md) | [Następna: Security Engineer](#) → (../security-engineer.md)

Zakres odpowiedzialności

Jako DevOps Engineer w projekcie IOC Service jesteś odpowiedzialny za:

- CI/CD pipeline (build, test, deploy)
 - Container orchestration (Docker Compose → opcjonalnie K8s)
 - Infrastructure as Code (IaC)
 - Monitoring i observability (Prometheus, Grafana, ELK)
 - Backup i disaster recovery
 - Scaling i performance
 - Security hardening (infrastruktura)
-

CI/CD Pipeline

Rekomendowany pipeline (GitHub Actions / GitLab CI)

```
graph LR
  COMMIT[Git Push] --> LINT[Lint & Format<br/>ruff, black, mypy]
  LINT --> TEST[Test<br/>pytest + coverage]
  TEST --> SECURITY[Security Scan<br/>bandit, safety, trivy]
  SECURITY --> BUILD[Docker Build<br/>Multi-stage]
  BUILD --> STAGING[Deploy Staging<br/>Docker Compose]
  STAGING --> E2E[E2E Tests<br/>Playwright]
  E2E --> APPROVE[Manual Approval]
  APPROVE --> PROD[Deploy Production<br/>Blue-Green]

  style SECURITY fill:#ff4444,color:#fff
  style PROD fill:#4CAF50,color:#fff
```

GitHub Actions Example

```

# .github/workflows/ci.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install ruff black mypy
      - run: ruff check app/ tests/
      - run: black --check app/ tests/
      - run: mypy app/

  test:
    runs-on: ubuntu-latest
    needs: lint
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_DB: ioc_test
          POSTGRES_PASSWORD: test
        ports: ["5432:5432"]
      redis:
        image: redis:7
        ports: ["6379:6379"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -e ".[dev]"
      - run: |
          pytest -v --cov=app --cov-report=xml \
            --cov-fail-under=80
      env:
        DATABASE_URL: postgresql://postgres:test@localhost:5432/ioc_test
        REDIS_URL: redis://localhost:6379/0
        SECRET_KEY: test-secret-key-minimum-32-characters-long

  security:
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4
      - run: pip install bandit safety
      - run: bandit -r app/ -ll
      - run: safety check
      - uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          scan-ref: .

  build:
    runs-on: ubuntu-latest

```

```
needs: security
steps:
  - uses: actions/checkout@v4
  - uses: docker/build-push-action@v5
    with:
      push: ${ github.ref == 'refs/heads/main' }
      tags: ioc-service:${ github.sha }
```

Docker Compose (Production)

```

# docker-compose.prod.yml
version: "3.8"

services:
  app:
    image: ioc-service:${VERSION:-latest}
    build:
      context: .
      dockerfile: Dockerfile
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      - DATABASE_URL=postgresql://ioc:${DB_PASSWORD}@postgres:5432/ioc_service
      - REDIS_URL=redis://redis:6379/0
      - SECRET_KEY_FILE=/run/secrets/secret_key
    secrets:
      - secret_key
      - db_password
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/healthz"]
      interval: 30s
      timeout: 5s
      retries: 3
    deploy:
      resources:
        limits:
          cpus: "2"
          memory: 2G

  worker:
    image: ioc-service:${VERSION:-latest}
    command: python -m app.worker
    restart: unless-stopped
    environment:
      - DATABASE_URL=postgresql://ioc:${DB_PASSWORD}@postgres:5432/ioc_service
      - REDIS_URL=redis://redis:6379/0
    secrets:
      - secret_key
      - db_password
      - api_keys
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    deploy:
      resources:
        limits:
          cpus: "1"
          memory: 1G

  postgres:
    image: postgres:16-alpine
    restart: unless-stopped
    volumes:
      - pg_data:/var/lib/postgresql/data
    environment:
      POSTGRES_DB: ioc_service
      POSTGRES_USER: ioc
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password

```

```

secrets:
  - db_password
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ioc"]
  interval: 10s
  timeout: 5s
  retries: 5

redis:
  image: redis:7-alpine
  restart: unless-stopped
  command: redis-server --maxmemory 512mb --maxmemory-policy allkeys-lru --
appendonly yes
  volumes:
    - redis_data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s

nginx:
  image: nginx:alpine
  restart: unless-stopped
  ports:
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./nginx/ssl:/etc/nginx/ssl:ro
  depends_on:
    - app

prometheus:
  image: prom/prometheus:latest
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prom_data:/prometheus

grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  volumes:
    - ./monitoring/grafana:/etc/grafana/provisioning/
    - grafana_data:/var/lib/grafana

volumes:
  pg_data:
  redis_data:
  prom_data:
  grafana_data:

secrets:
  secret_key:
    file: ./secrets/secret_key.txt
  db_password:
    file: ./secrets/db_password.txt
  api_keys:
    file: ./secrets/api_keys.txt

```

Monitoring & Observability

Prometheus Metrics (już zaimplementowane)

- `ioc_request_duration_seconds` — HTTP request latency
- `ioc_request_total` — Request count by endpoint, status
- `ioc_adapter_fetch_total` — Feed fetch operations
- `ioc_adapter_fetch_duration_seconds` — Feed fetch latency
- `ioc_indicators_total` — Total indicators gauge
- `ioc_circuit_breaker_state` — Circuit breaker per feed

Grafana Dashboards

1. **Overview Dashboard** — Request rate, error rate, latency p50/p95/p99
2. **Feed Health** — Per-feed success rate, circuit breaker state, items fetched
3. **Database** — Connection pool, query latency, disk usage
4. **Infrastructure** — CPU, memory, disk, network

Alerting Rules

Alert	Condition	Severity
HighErrorRate	Error rate >5% for 5min	● Critical
SlowResponses	p95 >500ms for 10min	● Warning
FeedDown	No successful fetch for 1h	● Warning
CircuitOpen	Circuit breaker open >15min	● Critical
DiskSpaceLow	Disk usage >85%	● Warning
HighMemory	Container memory >80%	● Warning
SSLExpiry	Certificate expires in <14d	● Warning

Backup & Disaster Recovery

Backup Strategy

```
#!/bin/bash
# scripts/backup.sh – Automated daily backup

BACKUP_DIR="/backups/ioc-service/${date +%Y-%m-%d}"
mkdir -p "$BACKUP_DIR"

# PostgreSQL backup
docker compose exec -T postgres pg_dump -U ioc ioc_service \
  | gzip > "$BACKUP_DIR/postgres.sql.gz"

# Redis backup
docker compose exec redis redis-cli BGSAVE
cp ./redis_data/dump.rdb "$BACKUP_DIR/redis.rdb"

# Configuration backup
tar czf "$BACKUP_DIR/config.tar.gz" \
  docker-compose.yml nginx/ monitoring/ config/

# Encrypt backup (GPG)
gpg --symmetric --cipher-algo AES256 "$BACKUP_DIR/postgres.sql.gz"
rm "$BACKUP_DIR/postgres.sql.gz"

# Retention: keep 30 days
find /backups/ioc-service -maxdepth 1 -mtime +30 -exec rm -rf {} \;
```

Recovery Targets

Metryka	Target	Uzasadnienie
RTO (Recovery Time Objective)	<30 min	Business criticality: medium
RPO (Recovery Point Objective)	<1h	Daily backup + WAL archiving
Backup frequency	Daily full + continuous WAL	Balance cost vs RPO
Backup retention	30 days	Compliance requirement
Recovery test frequency	Monthly	Verify backup integrity

Deployment Strategies

Blue-Green Deployment (recommended)

```
#!/bin/bash
# scripts/deploy-blue-green.sh

NEW_VERSION=$1
CURRENT=$(docker compose ps --format json | jq -r '.[0].Image')

# 1. Pull new version
docker pull ioc-service:$NEW_VERSION

# 2. Start green environment
VERSION=$NEW_VERSION docker compose -f docker-compose.green.yml up -d

# 3. Wait for health check
for i in $(seq 1 30); do
    if curl -sf http://localhost:8081/healthz; then
        echo "Green is healthy"
        break
    fi
    sleep 2
done

# 4. Switch traffic (nginx upstream)
sed -i "s/localhost:8080/localhost:8081/" nginx/upstream.conf
nginx -s reload

# 5. Stop blue (old version)
docker compose -f docker-compose.blue.yml down

echo "Deployed $NEW_VERSION (was $CURRENT)"
```

Kubernetes Migration Path (opcjonalny, v2.1+)

Jeśli zespół zdecyduje o migracji do K8s:

1. Helm chart z obecnego docker-compose
2. Horizontal Pod Autoscaler dla app pods
3. PersistentVolumeClaim dla PostgreSQL
4. Redis Sentinel / Cluster
5. Ingress controller zamiast Nginx

Security Hardening (Infrastructure)

CIS Docker Benchmark (key items)

- [x] Non-root user w Dockerfile
- [x] Read-only root filesystem (where possible)
- [] Docker Content Trust (image signing)
- [x] Resource limits (CPU, memory)
- [x] Health checks na wszystkich services
- [] Network segmentation (app, db, monitoring)
- [x] No secrets in environment variables (use Docker secrets)

Network segmentation

```
# docker-compose networks
networks:
  frontend: # nginx ↔ app
  backend:  # app ↔ postgres, redis
  monitoring: # prometheus ↔ grafana
```

[← Frontend Developer](#) (./frontend-developer.md) | [Następna: Security Engineer](#) → (./security-engineer.md)